

PRD NaikApa

Product Requirement Document untuk Aplikasi Android Rekomendasi Transportasi Multimoda

1. Ringkasan Produk

Nama aplikasi: NaikApa

Platform: Android native

Bahasa pengembangan: Kotlin

Database: SQLite lokal

Area layanan: Jabodetabek

Target pengguna: Mahasiswa dan pengguna transportasi umum

Jenis aplikasi: Aplikasi rekomendasi rute dan keputusan transportasi multimoda

NaikApa adalah aplikasi Android yang membantu pengguna menentukan pilihan transportasi paling sesuai berdasarkan lokasi awal, tujuan, moda yang tersedia, preferensi perjalanan, biaya, waktu, jarak jalan kaki, jumlah transit, dan laporan gangguan dari pengguna lain.

Berbeda dari aplikasi cek rute biasa, NaikApa tidak hanya menampilkan jalur perjalanan. Aplikasi ini memberikan rekomendasi utama, alternatif rute, skor kecocokan, alasan rekomendasi, estimasi waktu, estimasi biaya, estimasi BBM untuk kendaraan pribadi, serta peringatan gangguan apabila ada laporan dari pengguna.

Untuk rute transportasi umum, aplikasi menggunakan data GTFS TransJakarta, KRL, MRT, dan LRT. Untuk pencarian alamat, rute kendaraan pribadi, peta, dan visualisasi jalur, aplikasi menggunakan TomTom API dan TomTom Maps SDK.

2. Latar Belakang Masalah

Pengguna transportasi di Jabodetabek sering bingung memilih moda perjalanan yang paling cocok. Keputusan perjalanan tidak hanya bergantung pada titik awal dan tujuan, tetapi juga biaya, waktu tempuh, jumlah transit, jarak jalan kaki, ketersediaan kendaraan pribadi, dan kondisi perjalanan saat itu.

Contohnya, mahasiswa yang ingin pergi ke kampus mungkin ingin mencari rute paling hemat. Di kondisi lain, pengguna yang sedang terburu-buru mungkin lebih memilih rute tercepat walaupun biayanya lebih mahal. Pengguna yang memiliki motor atau mobil juga membutuhkan perbandingan apakah lebih baik langsung berkendara, naik transportasi umum, atau memakai kendaraan pribadi ke stasiun terdekat lalu melanjutkan perjalanan dengan transportasi umum.

Masalah lain yang muncul adalah gangguan perjalanan. Misalnya halte ramai, stasiun penuh, akses jalan kaki tidak nyaman, atau titik transportasi sedang bermasalah. Informasi seperti ini dapat memengaruhi keputusan pengguna, sehingga aplikasi perlu menyediakan fitur laporan gangguan dan rekomendasi yang lebih kontekstual.

3. Rumusan Masalah

1. Bagaimana membantu pengguna memilih moda transportasi yang paling sesuai berdasarkan kondisi perjalanan?
2. Bagaimana menggabungkan rute transportasi umum dan kendaraan pribadi dalam satu sistem rekomendasi?
3. Bagaimana memberikan rekomendasi berdasarkan prioritas seperti tercepat, terhemat, minim jalan kaki, dan minim transit?
4. Bagaimana menampilkan estimasi biaya, waktu, BBM, jarak jalan kaki, jumlah transit, serta alasan rekomendasi secara jelas?
5. Bagaimana memanfaatkan laporan gangguan dari pengguna agar rekomendasi perjalanan menjadi lebih relevan?
6. Bagaimana menerapkan database SQLite, fitur CRUD, dan multimedia dalam aplikasi transportasi berbasis Android?

4. Tujuan Produk

Tujuan aplikasi NaikApa adalah:

1. Membantu pengguna memilih moda transportasi paling sesuai untuk perjalanan di Jabodetabek.
2. Menggabungkan rute transportasi umum dan kendaraan pribadi dalam satu sistem rekomendasi.
3. Menampilkan rekomendasi rute berdasarkan prioritas tercepat, terhemat, minim jalan kaki, dan minim transit.
4. Memberikan perbandingan antara rute transportasi umum, kendaraan pribadi, dan kombinasi keduanya.
5. Menyediakan estimasi waktu, biaya, BBM, jarak jalan kaki, jumlah transit, dan skor kecocokan.
6. Menyediakan laporan gangguan berbasis kontribusi pengguna.
7. Menyimpan riwayat perjalanan, riwayat pencarian, data pengguna, kendaraan pribadi, dan perjalanan favorit.
8. Memenuhi kebutuhan tugas Pemrograman Mobile melalui penggunaan SQLite, fitur CRUD, dan multimedia.

5. Ruang Lingkup Produk

5.1 Scope MVP

Fitur yang masuk MVP:

1. Login dan register lokal menggunakan SQLite.
2. Home dengan input lokasi awal dan tujuan.
3. Pengambilan lokasi awal dari GPS perangkat.
4. Pencarian tujuan menggunakan TomTom Search API.
5. Pemilihan moda perjalanan.
6. Perhitungan rute transportasi umum menggunakan data GTFS.
7. Perhitungan rute kendaraan pribadi menggunakan TomTom Routing API.
8. Perhitungan rute gabungan, misalnya motor ke stasiun terdekat, lanjut KRL, lalu jalan kaki ke tujuan.
9. Pemilihan prioritas perjalanan.

10. Hasil rekomendasi berisi rekomendasi utama, alternatif, skor kecocokan, estimasi biaya, estimasi BBM, estimasi waktu, jarak jalan kaki, jumlah transit, alasan rekomendasi, dan peringatan gangguan.
11. Peta dan polyline rute menggunakan TomTom Maps SDK.
12. Detail langkah perjalanan.
13. Laporan gangguan dari pengguna.
14. Gangguan aktif otomatis selama 1 jam.
15. Gangguan memberi penalti ringan ke rute, bukan langsung memblokir rute.
16. CRUD laporan gangguan.
17. CRUD perjalanan favorit.
18. CRUD data pribadi pengguna.
19. Riwayat pencarian dan riwayat perjalanan.
20. Multimedia berupa foto laporan gangguan, ikon moda, peta, polyline, dan gambar pendukung.

5.2 Scope Lanjutan

Fitur yang tidak wajib untuk MVP, tetapi dapat dikembangkan setelah fitur utama selesai:

1. Integrasi ojek online.
2. Validasi laporan gangguan oleh admin.
3. Sistem reputasi laporan pengguna.
4. Integrasi GTFS Realtime jika tersedia.
5. Optimasi algoritma dengan A*.
6. Update data GTFS otomatis dari server.
7. Sinkronisasi akun online.
8. Estimasi tarif parkir untuk skenario park and ride.
9. Rekomendasi berbasis cuaca.
10. Notifikasi gangguan perjalanan.

6. Batasan Produk

1. Aplikasi dibuat untuk kebutuhan akademik Pemrograman Mobile.
2. Aplikasi menggunakan Android native Kotlin.
3. Database utama menggunakan SQLite lokal.
4. Login dan register disimpan secara lokal di SQLite.
5. API key TomTom disimpan di aplikasi untuk kebutuhan MVP akademik.
6. Data GTFS bersifat data transit nyata, tetapi sebagian data dapat berupa custom validasi manual berdasarkan sumber resmi.
7. Data KRL, MRT, dan LRT dapat menggunakan model headway atau estimasi jadwal apabila jadwal eksak belum tersedia.
8. Laporan gangguan berasal dari user dan langsung memengaruhi sistem sebagai penalti ringan selama 1 jam.
9. Tidak ada role admin pada MVP.
10. CRUD titik transportasi GTFS tidak masuk MVP karena titik transportasi berasal dari data GTFS. User hanya dapat melihat data titik, bukan mengubah data inti GTFS.
11. Ojek online tidak masuk MVP.
12. Aplikasi membutuhkan internet untuk fitur TomTom Search, TomTom Routing, dan TomTom Maps.

13. Aplikasi masih dapat menyimpan data lokal seperti user, favorit, riwayat, dan laporan di SQLite.

7. Target Pengguna

7.1 Mahasiswa

Mahasiswa membutuhkan rute yang hemat, mudah dipahami, dan cocok untuk perjalanan ke kampus, stasiun, halte, atau tempat umum. Mahasiswa juga cenderung mempertimbangkan biaya dan waktu.

7.2 Pengguna Transportasi Umum

Pengguna transportasi umum membutuhkan informasi rute multimoda, estimasi transit, jarak jalan kaki, waktu tempuh, dan alternatif jika ada gangguan.

7.3 Pengguna dengan Kendaraan Pribadi

Pengguna yang memiliki motor atau mobil membutuhkan perbandingan apakah lebih efisien berkendara langsung, naik transportasi umum, atau menggunakan kendaraan pribadi menuju titik transportasi terdekat.

8. Value Proposition

NaikApa memiliki nilai utama sebagai aplikasi rekomendasi keputusan transportasi, bukan hanya aplikasi pencari rute.

Nilai pembeda NaikApa:

1. Memberikan jawaban “sebaiknya naik apa”, bukan hanya “lewat mana”.
2. Menggabungkan GTFS transportasi umum dan TomTom untuk kendaraan pribadi.
3. Mendukung rute gabungan seperti motor ke stasiun, lanjut KRL, lalu jalan kaki.
4. Menampilkan skor kecocokan dan alasan rekomendasi.
5. Mempertimbangkan laporan gangguan dari pengguna.
6. Memiliki preferensi perjalanan yang mudah dipahami.
7. Tetap memiliki aspek database, CRUD, dan multimedia sesuai kebutuhan tugas.

9. Fitur Produk

9.1 Splash Screen

Deskripsi:

Halaman awal saat aplikasi dibuka.

Kebutuhan:

1. Menampilkan logo NaikApa.
2. Menampilkan tagline singkat.

3. Setelah beberapa detik, user diarahkan ke Login atau Home jika sudah login.

9.2 Login

Deskripsi:

User masuk ke aplikasi menggunakan akun lokal.

Kebutuhan:

1. Input email.
2. Input password.
3. Validasi email dan password dari SQLite.
4. Tombol masuk.
5. Link ke Register.

Catatan:

Login ini bersifat lokal untuk kebutuhan akademik, bukan autentikasi cloud.

9.3 Register

Deskripsi:

User membuat akun lokal.

Kebutuhan:

1. Input nama.
2. Input email.
3. Input password.
4. Input konfirmasi password.
5. Pilihan apakah user memiliki motor.
6. Pilihan apakah user memiliki mobil.
7. Data disimpan ke SQLite.

9.4 Home

Deskripsi:

Halaman utama untuk mencari rute.

Konsep UI:

Mengikuti gaya aplikasi transportasi modern, yaitu panel pencarian dengan pilihan moda, pilihan prioritas, input asal dan tujuan, tombol cari rute, ringkasan hasil, dan tampilan peta.

Komponen:

1. Pilihan moda.
2. Pilihan prioritas.
3. Input lokasi asal.
4. Tombol gunakan lokasi GPS.

5. Input tujuan dengan search TomTom.
6. Tombol cari rute.
7. Ringkasan rute.
8. Map preview.

9.5 Pilih Moda

Moda yang tersedia:

1. Campur semua.
2. TransJakarta.
3. KRL.
4. MRT.
5. LRT.
6. Transportasi umum saja.
7. Motor.
8. Mobil.
9. Kendaraan pribadi saja.

Aturan:

1. Jika user memilih TransJakarta, KRL, MRT, atau LRT, sistem hanya mencari rute dari moda tersebut.
2. Jika user memilih transportasi umum saja, sistem mencari kombinasi TransJakarta, KRL, MRT, LRT, dan jalan kaki.
3. Jika user memilih kendaraan pribadi saja, sistem mencari rute motor atau mobil via TomTom.
4. Jika user memilih campur semua, sistem dapat membandingkan dan menggabungkan transportasi umum, kendaraan pribadi, dan jalan kaki.

9.6 Pilih Prioritas

Prioritas yang tersedia:

1. Tercepat.
2. Terhemat.
3. Minim jalan kaki.
4. Minim transit.

Aturan:

1. Tercepat mengutamakan waktu tempuh.
2. Terhemat mengutamakan biaya transportasi dan estimasi BBM.
3. Minim jalan kaki mengutamakan rute dengan total jarak jalan kaki paling pendek.
4. Minim transit mengutamakan rute dengan jumlah perpindahan moda paling sedikit.

9.7 Search Lokasi dan Tujuan

Deskripsi:

User dapat mencari tujuan berupa halte, stasiun, atau alamat umum.

Kebutuhan:

1. Untuk halte dan stasiun, aplikasi mengambil data dari GTFS lokal.
2. Untuk alamat atau nama daerah, aplikasi menggunakan TomTom Search API.
3. Hasil pencarian menampilkan nama lokasi dan alamat.
4. User memilih salah satu hasil sebagai tujuan.
5. Sistem menyimpan riwayat pencarian ke SQLite.

9.8 Rekomendasi Rute

Deskripsi:

Fitur utama untuk menghitung dan menampilkan rekomendasi perjalanan.

Input:

1. Lokasi awal.
2. Lokasi tujuan.
3. Moda yang dipilih.
4. Prioritas.
5. Data kendaraan pribadi user.
6. Data GTFS.
7. Data TomTom.
8. Data laporan gangguan aktif.

Output:

1. Rekomendasi utama.
2. Alternatif 1.
3. Alternatif 2.
4. Skor kecocokan.
5. Estimasi biaya.
6. Estimasi BBM.
7. Estimasi waktu.
8. Jarak jalan kaki.
9. Jumlah transit.
10. Alasan rekomendasi.
11. Peringatan gangguan.
12. Langkah perjalanan.
13. Polyline pada peta.

9.9 Rute Gabungan Kendaraan Pribadi dan Transportasi Umum

Deskripsi:

Sistem dapat memprediksi skenario user memakai motor atau mobil ke titik transportasi terdekat, lalu melanjutkan perjalanan dengan transportasi umum.

Contoh:

1. Lokasi user ke stasiun terdekat menggunakan motor.
2. Stasiun terdekat ke stasiun tujuan menggunakan KRL.
3. Stasiun tujuan ke lokasi akhir dengan jalan kaki.

Kebutuhan:

1. Sistem mencari titik GTFS terdekat dari lokasi user.
2. Sistem menghitung rute kendaraan pribadi dari lokasi user ke titik GTFS terdekat menggunakan TomTom.
3. Sistem menghitung rute transit dari titik GTFS awal ke titik GTFS akhir menggunakan Dijkstra.
4. Sistem menghitung jarak jalan kaki dari titik transportasi akhir ke tujuan.
5. Sistem menggabungkan estimasi waktu, biaya, BBM, jalan kaki, dan transit.
6. Sistem memberi skor kecocokan untuk hasil gabungan.

9.10 Detail Rute

Deskripsi:

Menampilkan rincian perjalanan dari rekomendasi yang dipilih.

Isi halaman:

1. Nama rute.
2. Moda yang digunakan.
3. Estimasi waktu.
4. Estimasi biaya.
5. Estimasi BBM.
6. Total jarak.
7. Total jalan kaki.
8. Jumlah transit.
9. Skor kecocokan.
10. Alasan rekomendasi.
11. Peringatan gangguan.
12. Daftar langkah perjalanan.

Contoh langkah:

1. Mulai dari lokasi saat ini.
2. Berkendara motor ke Stasiun Tangerang.
3. Naik KRL menuju Stasiun Tanah Abang.
4. Transit ke MRT atau TransJakarta jika diperlukan.
5. Jalan kaki ke lokasi tujuan.

9.11 Peta Rute

Deskripsi:

Menampilkan jalur rute pada peta.

Kebutuhan:

1. Menggunakan TomTom Maps SDK.
2. Menampilkan marker lokasi awal.
3. Menampilkan marker tujuan.
4. Menampilkan marker halte atau stasiun transit.
5. Menampilkan polyline rute kendaraan pribadi.
6. Menampilkan polyline atau garis rute transportasi umum jika data shape tersedia.
7. Menampilkan mode peta dasar.

9.12 Laporan Gangguan

Deskripsi:

User dapat melaporkan gangguan pada rute, halte, stasiun, atau titik perjalanan.

Kategori laporan:

1. Halte atau stasiun penuh.
2. Antrean panjang.
3. Titik terlalu sepi.
4. Akses jalan kaki sulit.
5. Banjir.
6. Gangguan perjalanan.
7. Kendala lain.

Data laporan:

1. User pelapor.
2. Titik transportasi terkait.
3. Kategori gangguan.
4. Deskripsi.
5. Foto.
6. Waktu laporan.
7. Status aktif.
8. Waktu kedaluwarsa.

Aturan:

1. Laporan langsung masuk ke sistem.
2. Laporan aktif selama 1 jam.
3. Laporan memberi penalti ringan pada rute terkait.
4. Laporan muncul sebagai peringatan di hasil rekomendasi, detail rute, dan halaman status gangguan.
5. User dapat membuat, melihat, mengedit, dan menghapus laporan miliknya.

9.13 Status Gangguan

Deskripsi:

Halaman untuk melihat gangguan aktif.

Kebutuhan:

1. Menampilkan daftar laporan gangguan aktif.
2. Menampilkan lokasi gangguan.
3. Menampilkan kategori.
4. Menampilkan foto jika ada.
5. Menampilkan waktu laporan.
6. Menampilkan sisa waktu aktif.
7. Menampilkan dampak pada rekomendasi.

9.14 Perjalanan Favorit

Deskripsi:

User dapat menyimpan rute yang sering digunakan.

Kebutuhan:

1. Simpan rute dari hasil rekomendasi.
2. Lihat daftar favorit.
3. Edit nama favorit.
4. Hapus favorit.
5. Jalankan ulang pencarian dari favorit.

9.15 Riwayat Pencarian

Deskripsi:

Menyimpan lokasi yang pernah dicari user.

Data:

1. Keyword pencarian.
2. Lokasi terpilih.
3. Koordinat.
4. Waktu pencarian.

9.16 Riwayat Perjalanan

Deskripsi:

Menyimpan hasil rute yang pernah dipilih user.

Data:

1. Lokasi awal.
2. Tujuan.
3. Moda.
4. Prioritas.
5. Rekomendasi terpilih.
6. Estimasi waktu.
7. Estimasi biaya.
8. Skor kecocokan.
9. Tanggal perjalanan.

9.17 Profil Pengguna

Deskripsi:

User dapat mengelola data pribadi.

Data:

1. Nama.
2. Email.
3. Password.
4. Status memiliki motor.
5. Status memiliki mobil.

CRUD:

1. Read profil.
2. Update profil.
3. Delete akun lokal.
4. Update data kendaraan pribadi.

10. Algoritma Rekomendasi

10.1 Data Sumber

Data yang digunakan:

1. GTFS TransJakarta.
2. GTFS KRL.
3. GTFS MRT.
4. GTFS LRT.
5. TomTom Search API.
6. TomTom Routing API.
7. TomTom Maps SDK.
8. SQLite lokal.
9. Laporan gangguan aktif.

Data transit dimuat ke SQLite dan dibangun menjadi graph berarah berbobot di memori. Pada versi Android, data GTFS dan logika Dijkstra dibundel langsung ke aplikasi memakai SQLite native.

10.2 Dijkstra Multimodal

Untuk rute transportasi umum, sistem menggunakan algoritma Dijkstra multimodal.

Node:

1. Halte TransJakarta.
2. Stasiun KRL.
3. Stasiun MRT.
4. Stasiun LRT.
5. Titik transfer jalan kaki.

Edge:

1. Perpindahan antar halte atau stasiun dalam rute yang sama.
2. Transfer antar moda.
3. Jalan kaki antar titik transportasi.
4. Koneksi kendaraan pribadi ke titik transportasi terdekat untuk mode campur semua.

10.3 Walking Transfer

Sistem membuat koneksi jalan kaki antar titik transportasi yang berdekatan.

Aturan:

1. Walking transfer dibuat antar titik dari agensi berbeda.
2. Jarak dihitung menggunakan Haversine.
3. Ambang walking transfer default 350 meter.
4. Durasi jalan kaki dihitung dari asumsi kecepatan jalan kaki sekitar 4 km per jam.

10.4 Rute Kendaraan Pribadi

Untuk motor dan mobil:

1. Sistem menggunakan TomTom Routing API.
2. Mode motor memakai travel mode motorcycle.
3. Mode mobil memakai travel mode car.
4. Sistem meminta rute utama dan alternatif.
5. Untuk mobil, sistem dapat menyediakan opsi hindari tol.
6. Estimasi BBM dihitung dari jarak.

10.5 Rute Campur Semua

Mode campur semua mencari opsi terbaik dari:

1. Transportasi umum penuh.
2. Motor penuh.
3. Mobil penuh.
4. Motor ke stasiun atau halte terdekat, lalu transportasi umum.
5. Mobil ke stasiun atau halte terdekat, lalu transportasi umum.
6. Transportasi umum lalu jalan kaki ke tujuan.

Tahapan:

1. Ambil lokasi user dari GPS.
2. Ambil tujuan dari TomTom Search atau data GTFS.
3. Cari titik transportasi terdekat dari lokasi user.
4. Cari titik transportasi terdekat dari tujuan.
5. Hitung rute kendaraan pribadi ke titik transportasi awal jika diperlukan.
6. Hitung rute transit dengan Dijkstra.
7. Hitung jalan kaki dari titik akhir ke tujuan jika diperlukan.
8. Gabungkan semua segmen.
9. Hitung skor kecocokan.
10. Urutkan hasil berdasarkan prioritas user.

11. Logika Prioritas

11.1 Tercepat

Bobot utama:

1. Waktu perjalanan.
2. Waktu kendaraan pribadi.
3. Waktu transit.
4. Waktu jalan kaki.
5. Penalti gangguan.

Cocok untuk user yang ingin sampai secepat mungkin.

11.2 Terhemat

Bobot utama:

1. Tarif transportasi umum.
2. Estimasi BBM.
3. Biaya kombinasi perjalanan.
4. Penalti biaya jika memakai mobil atau motor terlalu jauh.

Cocok untuk mahasiswa atau user dengan budget terbatas.

11.3 Minim Jalan Kaki

Bobot utama:

1. Total jarak jalan kaki.
2. Jumlah walking transfer.
3. Penalti besar untuk segmen jalan kaki yang panjang.
4. Penalti untuk rute dengan akses jalan kaki yang terkena laporan gangguan.

Cocok untuk user yang membawa barang, sedang lelah, atau ingin perjalanan lebih nyaman.

11.4 Minim Transit

Bobot utama:

1. Jumlah perpindahan moda.
2. Jumlah ganti kendaraan.
3. Jumlah walking transfer.
4. Waktu sebagai tiebreaker.

Cocok untuk user yang ingin rute sederhana dan tidak banyak pindah moda.

12. Skor Kecocokan

Skor kecocokan adalah nilai persentase yang menunjukkan seberapa cocok sebuah rute dengan prioritas user.

Rentang skor:

1. 85 sampai 100: sangat cocok.
2. 70 sampai 84: cukup cocok.
3. 55 sampai 69: bisa dipakai, tetapi ada kekurangan.
4. Di bawah 55: kurang direkomendasikan.

Faktor skor:

1. Waktu.
2. Biaya.
3. Estimasi BBM.
4. Jarak jalan kaki.
5. Jumlah transit.
6. Gangguan aktif.
7. Kesesuaian dengan moda pilihan.
8. Ketersediaan kendaraan pribadi user.

Contoh rumus sederhana:

```
skor = 100
skor -= penalti_waktu
skor -= penalti_biaya
skor -= penalti_jalan_kaki
skor -= penalti_transit
skor -= penalti_gangguan
skor += bonus_sesuai_prioritas
```

Contoh bobot:

1. Tercepat: waktu 45%, biaya 15%, jalan kaki 15%, transit 15%, gangguan 10%.
2. Terhemat: biaya 45%, waktu 20%, jalan kaki 15%, transit 10%, gangguan 10%.
3. Minim jalan kaki: jalan kaki 45%, waktu 20%, biaya 15%, transit 10%, gangguan 10%.
4. Minim transit: transit 45%, waktu 20%, biaya 15%, jalan kaki 10%, gangguan 10%.

13. Alasan Rekomendasi

Setiap hasil rekomendasi wajib memiliki alasan yang bisa dipahami user.

Contoh alasan:

1. "Rute ini dipilih karena memiliki waktu tempuh paling singkat dibanding alternatif lain."
2. "Rute ini lebih hemat karena menggunakan transportasi umum dan hanya memakai motor untuk menuju stasiun terdekat."
3. "Rute ini cocok karena jarak jalan kaki lebih pendek."
4. "Rute ini memiliki transit lebih sedikit dibanding alternatif lain."
5. "Rute ini tetap ditampilkan, tetapi terdapat laporan gangguan aktif pada salah satu titik perjalanan."

14. Database SQLite

14.1 users

Menyimpan data pengguna lokal.

Field:

1. id_user
2. nama
3. email
4. password
5. has_motor
6. has_car
7. created_at

14.2 user_profiles

Menyimpan detail profil tambahan.

Field:

1. id_profile
2. id_user
3. default_mode
4. default_priority
5. home_lat
6. home_lon
7. home_label

14.3 gtfs_stops

Menyimpan halte dan stasiun dari GTFS.

Field:

1. stop_id
2. stop_name
3. stop_lat
4. stop_lon
5. agency_id
6. stop_type

14.4 gtfs_routes

Menyimpan rute GTFS.

Field:

1. route_id
2. agency_id
3. route_short_name
4. route_long_name
5. route_color
6. route_text_color

14.5 gtfs_trips

Menyimpan trip GTFS.

Field:

1. trip_id

2. route_id
3. service_id
4. direction_id

14.6 gtfs_stop_times

Menyimpan waktu kedatangan dan keberangkatan.

Field:

1. trip_id
2. arrival_time
3. departure_time
4. stop_id
5. stop_sequence

14.7 saved_trips

Menyimpan perjalanan favorit.

Field:

1. id_saved
2. id_user
3. nama_perjalanan
4. origin_name
5. origin_lat
6. origin_lon
7. destination_name
8. destination_lat
9. destination_lon
10. mode
11. priority
12. catatan
13. created_at

14.8 search_history

Menyimpan riwayat pencarian lokasi.

Field:

1. id_search
2. id_user
3. keyword
4. selected_name
5. selected_address
6. selected_lat

7. selected_lon
8. searched_at

14.9 route_history

Menyimpan riwayat rekomendasi rute.

Field:

1. id_history
2. id_user
3. origin_name
4. destination_name
5. mode
6. priority
7. recommendation_summary
8. score
9. estimated_time
10. estimated_cost
11. estimated_bbm
12. walking_distance
13. transit_count
14. created_at

14.10 disruption_reports

Menyimpan laporan gangguan.

Field:

1. id_report
2. id_user
3. stop_id
4. route_id
5. category
6. description
7. photo_path
8. impact_level
9. status
10. created_at
11. expired_at

14.11 route_cache

Opsional untuk menyimpan hasil rute sementara.

Field:

1. id_cache
2. origin_lat
3. origin_lon
4. destination_lat
5. destination_lon
6. mode
7. priority
8. result_json
9. created_at

15. CRUD

15.1 CRUD Data Pengguna

Create:

User membuat akun.

Read:

User melihat profil.

Update:

User mengubah nama, email, password, dan status kendaraan pribadi.

Delete:

User menghapus akun lokal.

15.2 CRUD Laporan Gangguan

Create:

User membuat laporan gangguan dengan foto.

Read:

User melihat daftar gangguan aktif.

Update:

User mengedit laporan miliknya.

Delete:

User menghapus laporan miliknya.

15.3 CRUD Perjalanan Favorit

Create:

User menyimpan rute favorit.

Read:

User melihat daftar favorit.

Update:

User mengubah nama atau catatan favorit.

Delete:

User menghapus favorit.

15.4 CRUD Riwayat

Create:

Sistem menyimpan riwayat pencarian dan perjalanan.

Read:

User melihat riwayat.

Update:

Tidak wajib.

Delete:

User menghapus riwayat tertentu atau semua riwayat.

16. Multimedia

Multimedia yang digunakan:

1. Foto laporan gangguan dari kamera.
2. Foto laporan gangguan dari galeri.
3. Ikon moda transportasi.
4. Marker peta.
5. Polyline rute.
6. Banner atau ilustrasi halaman home.
7. Logo aplikasi.
8. Visual peta dari TomTom Maps SDK.

Kebutuhan kamera:

1. Aplikasi meminta izin kamera.
2. User dapat mengambil foto untuk laporan.
3. Foto disimpan sebagai file lokal.
4. Path foto disimpan di SQLite.

Kebutuhan galeri:

1. Aplikasi meminta izin akses media.
2. User dapat memilih gambar dari galeri.

3. Path gambar disimpan di SQLite.

17. Halaman Aplikasi

17.1 MVP Wajib

1. Splash Screen.
2. Login.
3. Register.
4. Home.
5. Search Lokasi.
6. Hasil Rekomendasi.
7. Detail Rute.
8. Peta Rute.
9. Status Gangguan.
10. Tambah Laporan Gangguan.
11. Detail Laporan Gangguan.
12. Perjalanan Favorit.
13. Riwayat Pencarian.
14. Riwayat Perjalanan.
15. Profil Pengguna.

17.2 Tambahan Jika Sempat

1. Pengaturan default prioritas.
2. Detail titik transportasi.
3. Detail fasilitas stasiun atau halte.
4. Filter hindari tol.
5. Filter hanya rute tanpa gangguan.
6. Mode gelap dan terang.
7. Tutorial onboarding.

18. Alur Pengguna Utama

18.1 Alur Cari Rute

1. User membuka aplikasi.
2. User login atau register.
3. User masuk ke Home.
4. User memilih moda.
5. User memilih prioritas.
6. User menggunakan GPS sebagai lokasi awal.
7. User mencari tujuan.
8. User menekan tombol cari rute.
9. Sistem mengambil data GTFS, TomTom, profil user, dan laporan gangguan.
10. Sistem menghitung beberapa kandidat rute.
11. Sistem menghitung skor kecocokan.
12. Sistem menampilkan rekomendasi utama dan alternatif.

13. User membuka detail rute.
14. User melihat langkah perjalanan dan peta.
15. User dapat menyimpan rute ke favorit.

18.2 Alur Laporan Gangguan

1. User membuka detail rute atau halaman status gangguan.
2. User memilih buat laporan.
3. User memilih titik transportasi atau rute terkait.
4. User memilih kategori gangguan.
5. User menulis deskripsi.
6. User menambahkan foto dari kamera atau galeri.
7. User mengirim laporan.
8. Sistem menyimpan laporan ke SQLite.
9. Sistem mengaktifkan laporan selama 1 jam.
10. Sistem memberi penalti ringan pada rute terkait.

19. Kebutuhan Teknis

19.1 Android

1. Kotlin.
2. Minimum SDK disesuaikan dengan kebutuhan TomTom Maps SDK.
3. SQLite untuk database lokal.
4. TomTom Maps SDK untuk peta.
5. TomTom Search API untuk pencarian alamat.
6. TomTom Routing API untuk motor dan mobil.
7. GPS perangkat untuk lokasi awal.
8. Permission lokasi, internet, kamera, dan media.

19.2 Data GTFS

Data yang digunakan:

1. TransJakarta.
2. KRL.
3. MRT.
4. LRT.

Strategi:

1. GTFS diimpor menjadi SQLite.
2. File SQLite dibundel ke aplikasi.
3. Saat pertama kali aplikasi dibuka, database disalin ke internal storage.
4. Graph rute dibangun di memori saat aplikasi membutuhkan pencarian rute.
5. Jika performa berat, graph dapat di-cache.

19.3 API

TomTom Search:

1. Digunakan untuk mencari lokasi tujuan berdasarkan keyword.
2. Digunakan untuk mencari alamat bebas di Jabodetabek.

TomTom Routing:

1. Digunakan untuk rute motor.
2. Digunakan untuk rute mobil.
3. Digunakan untuk menghitung akses kendaraan pribadi ke stasiun atau halte terdekat.

TomTom Maps:

1. Digunakan untuk menampilkan peta.
2. Digunakan untuk menampilkan marker dan polyline.

20. Risiko dan Mitigasi

20.1 Risiko Porting Dijkstra ke Kotlin Cukup Berat

Mitigasi:

1. Mulai dari fitur rute transportasi umum sederhana.
2. Batasi kandidat titik awal dan akhir ke titik terdekat.
3. Cache graph di memori.
4. Gunakan data subset dulu saat pengujian.

20.2 Risiko Database GTFS Besar

Mitigasi:

1. Gunakan pre-built SQLite.
2. Hanya simpan field yang dibutuhkan.
3. Buat index pada stop_id, route_id, trip_id, dan stop_sequence.

20.3 Risiko API Key TomTom Terekspos

Mitigasi:

1. Untuk MVP akademik, API key boleh disimpan di aplikasi.
2. Untuk produksi, pindahkan API key ke backend.

20.4 Risiko Rute Gabungan Terlalu Kompleks

Mitigasi:

1. Batasi pencarian titik transportasi terdekat dalam radius tertentu.
2. Ambil beberapa kandidat terdekat saja.
3. Untuk MVP, prioritaskan hasil yang dapat dihitung stabil.

20.5 Risiko Laporan Gangguan Palsu

Mitigasi:

1. Efek gangguan hanya penalti ringan.
2. Gangguan aktif hanya 1 jam.
3. Riwayat laporan tetap tersimpan.

21. Kriteria Keberhasilan MVP

Aplikasi dianggap berhasil jika:

1. User dapat register dan login.
2. User dapat menggunakan GPS sebagai lokasi awal.
3. User dapat mencari tujuan dengan TomTom Search.
4. User dapat memilih moda dan prioritas.
5. Aplikasi dapat menghitung rute transportasi umum berbasis GTFS.
6. Aplikasi dapat menghitung rute motor dan mobil berbasis TomTom.
7. Aplikasi dapat menampilkan rute campuran kendaraan pribadi dan transportasi umum.
8. Aplikasi dapat menampilkan rekomendasi utama dan alternatif.
9. Aplikasi dapat menampilkan skor kecocokan dan alasan rekomendasi.
10. Aplikasi dapat menampilkan peta, marker, dan polyline.
11. User dapat membuat laporan gangguan dengan foto.
12. Gangguan memengaruhi rekomendasi sebagai penalti ringan.
13. User dapat menyimpan perjalanan favorit.
14. User dapat melihat riwayat pencarian dan perjalanan.
15. Aplikasi memenuhi kebutuhan database, CRUD, dan multimedia.

22. Kesimpulan

NaikApa adalah aplikasi Android native berbasis Kotlin yang membantu pengguna memilih transportasi terbaik di Jabodetabek. Aplikasi ini menggunakan GTFS untuk transportasi umum, TomTom API untuk pencarian lokasi dan kendaraan pribadi, SQLite untuk penyimpanan data lokal, serta sistem rekomendasi berbasis prioritas pengguna.

MVP NaikApa cukup kuat untuk tugas karena tidak hanya menampilkan rute, tetapi juga memberi rekomendasi utama, alternatif, skor kecocokan, alasan pemilihan, estimasi biaya, estimasi BBM, jarak jalan kaki, jumlah transit, peta rute, laporan gangguan, riwayat, favorit, dan multimedia. Hal ini membuat NaikApa lebih menonjol dibanding aplikasi CRUD biasa maupun aplikasi cek rute sederhana.